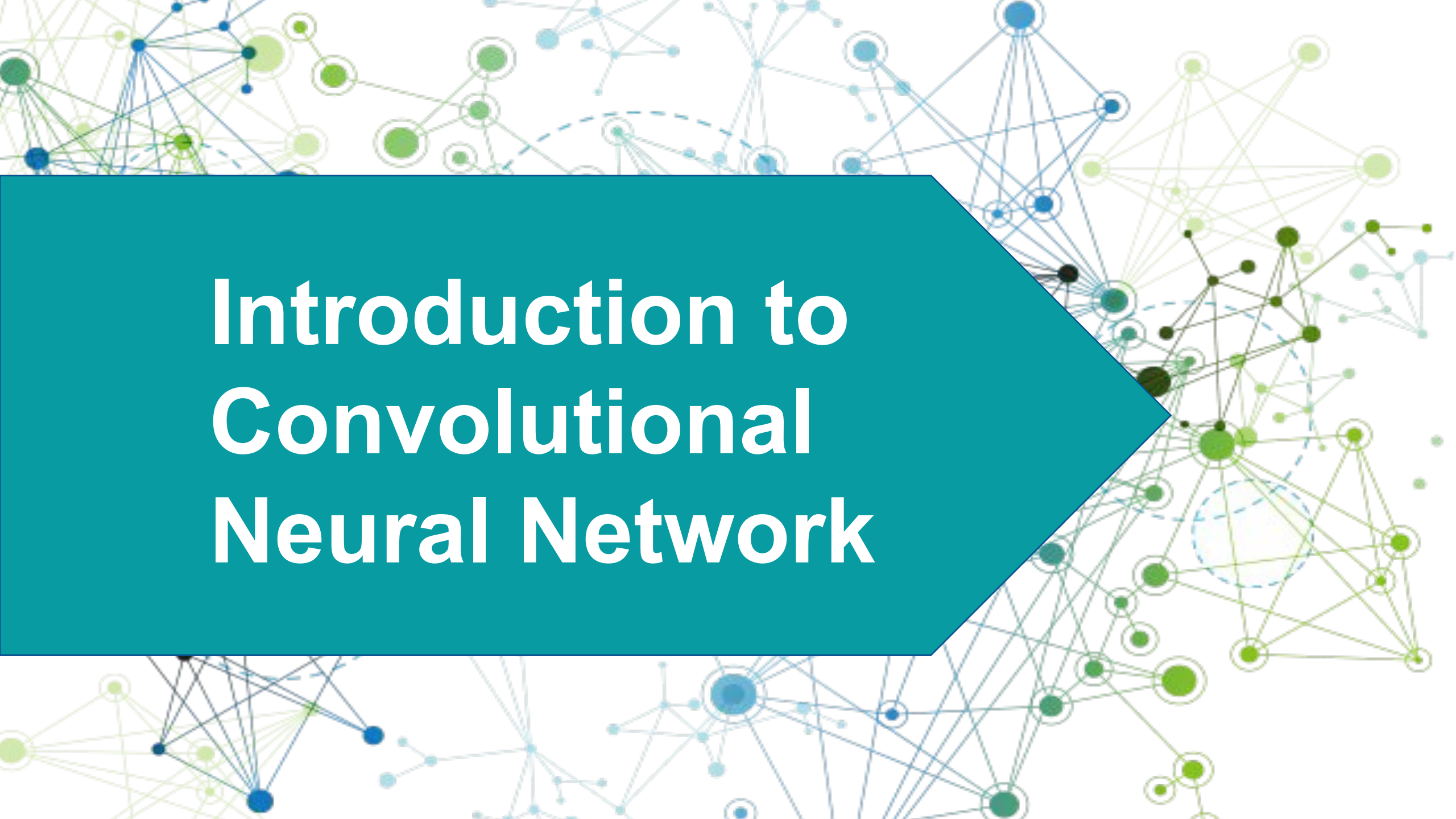
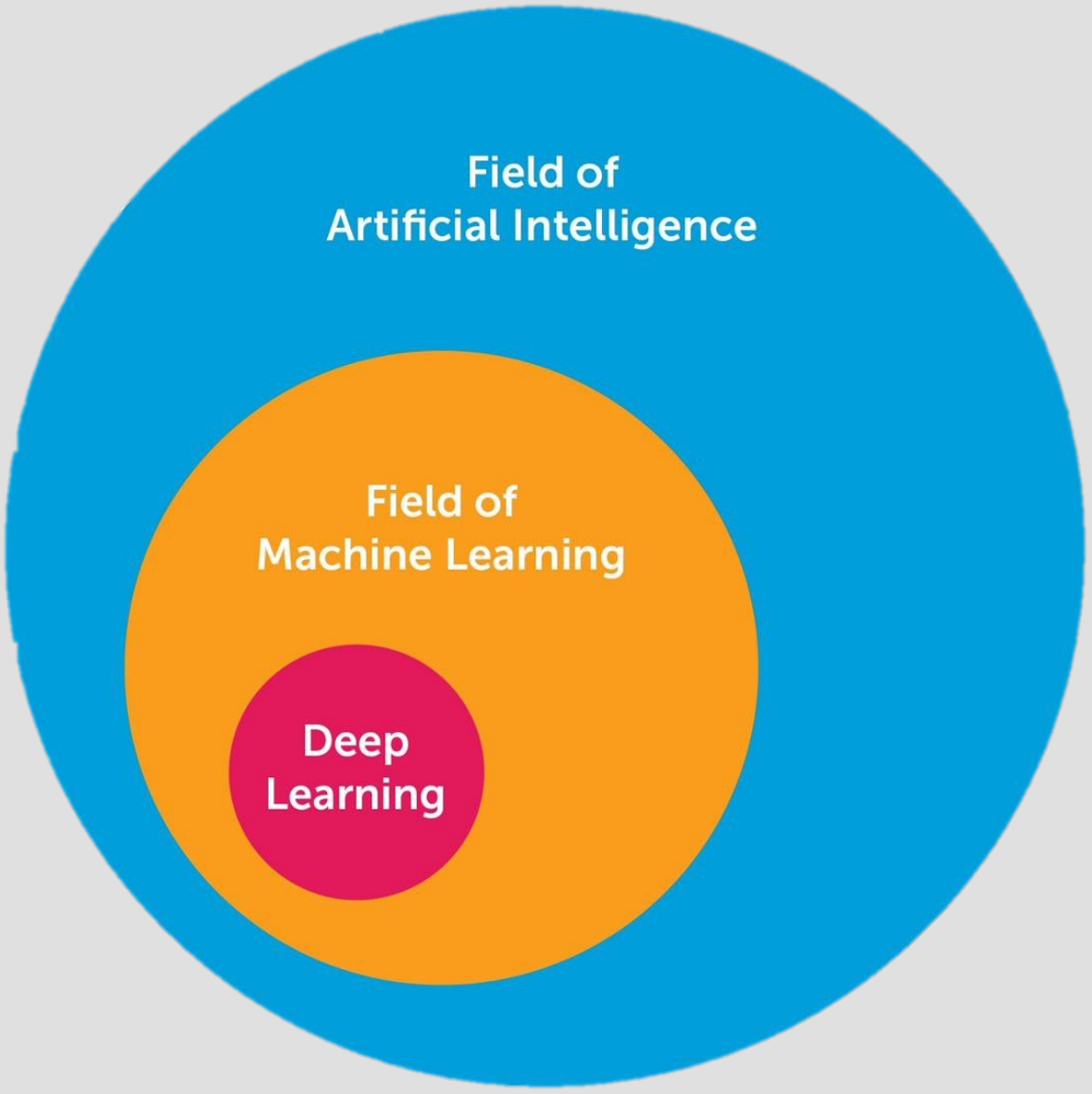




Introduction to Convolutional Neural Network

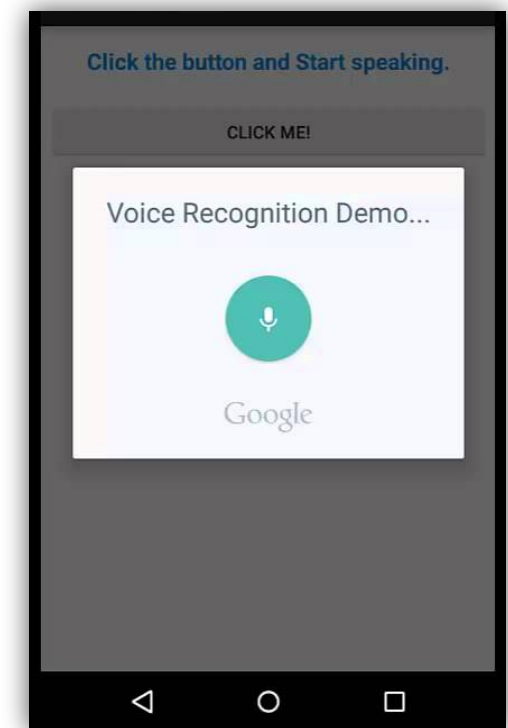
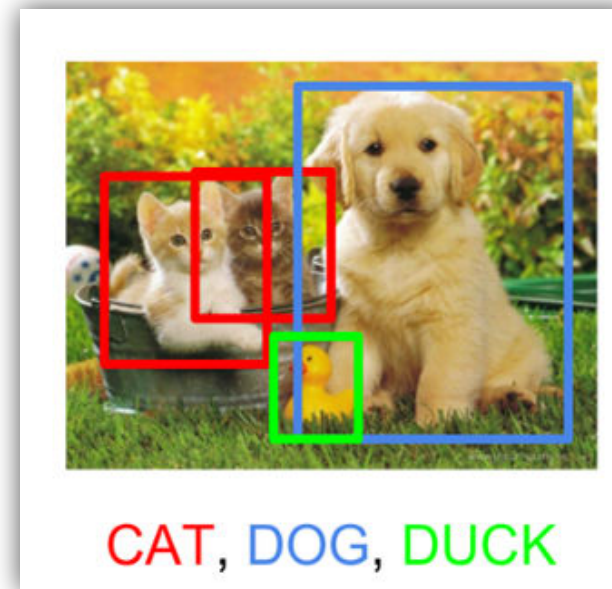


A Venn diagram illustrating the relationship between three fields of study. It consists of three concentric circles. The outermost circle is blue and labeled 'Field of Artificial Intelligence'. Inside it is an orange circle labeled 'Field of Machine Learning'. Inside the orange circle is a smaller pink circle labeled 'Deep Learning'. To the left of the circles is a vertical black line.

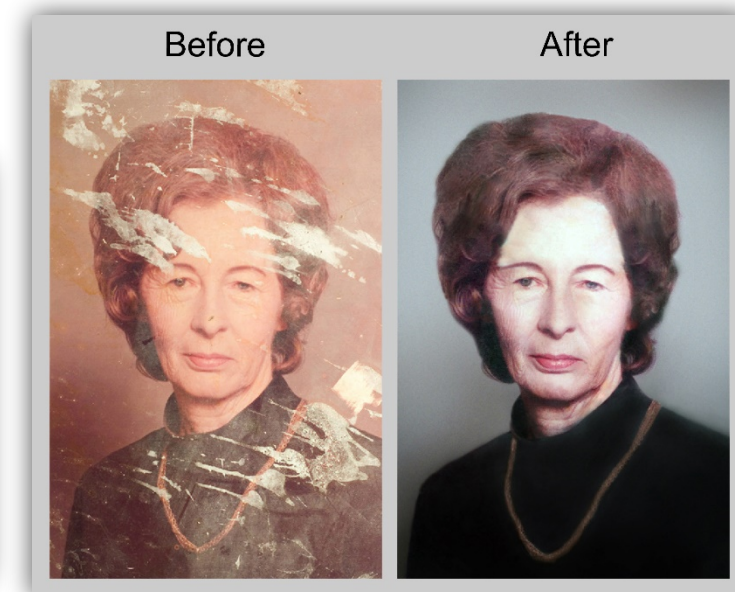
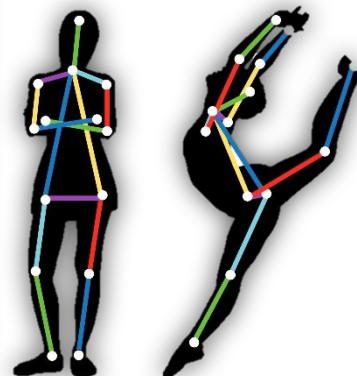
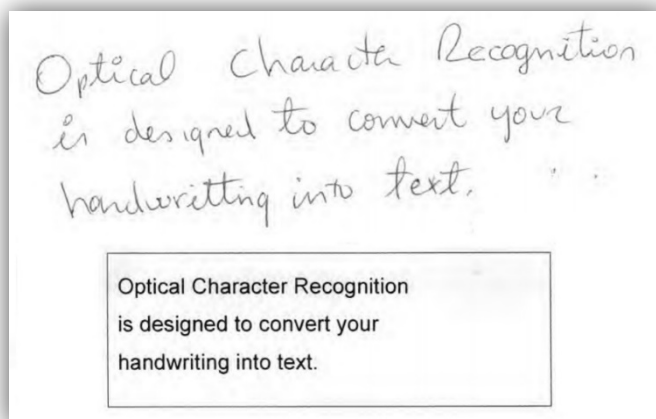
Field of
Artificial Intelligence

Field of
Machine Learning

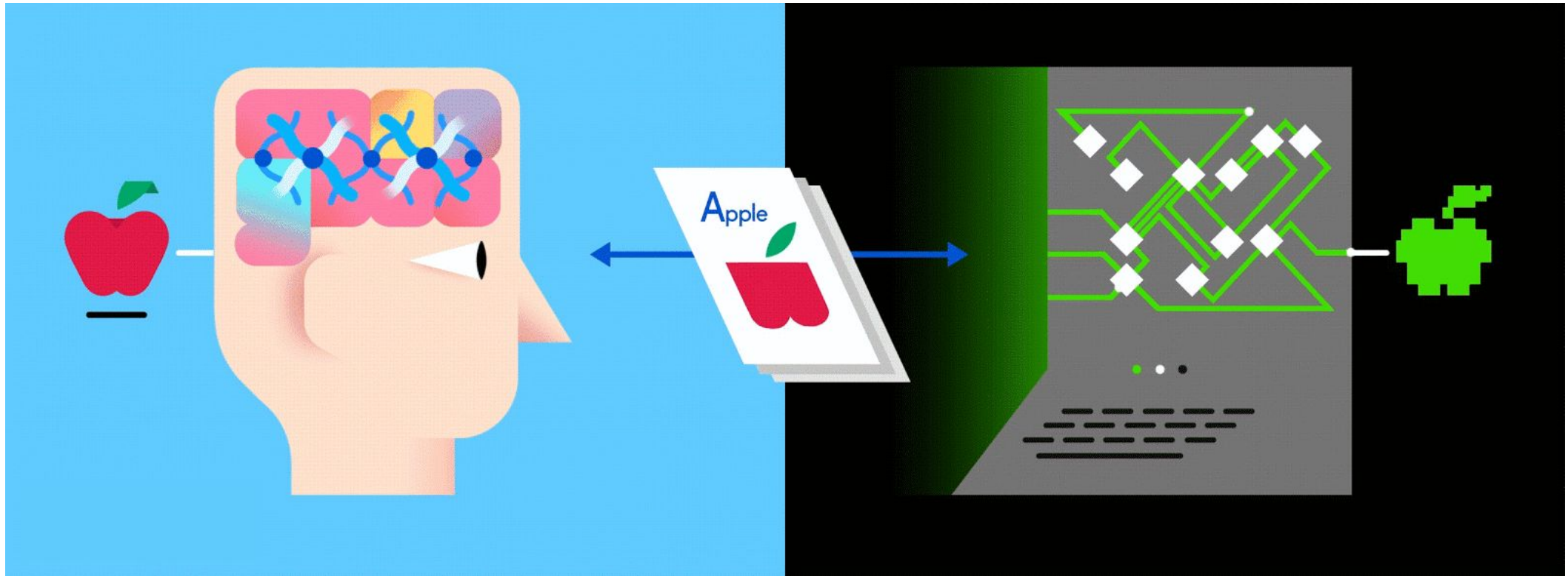
Deep
Learning



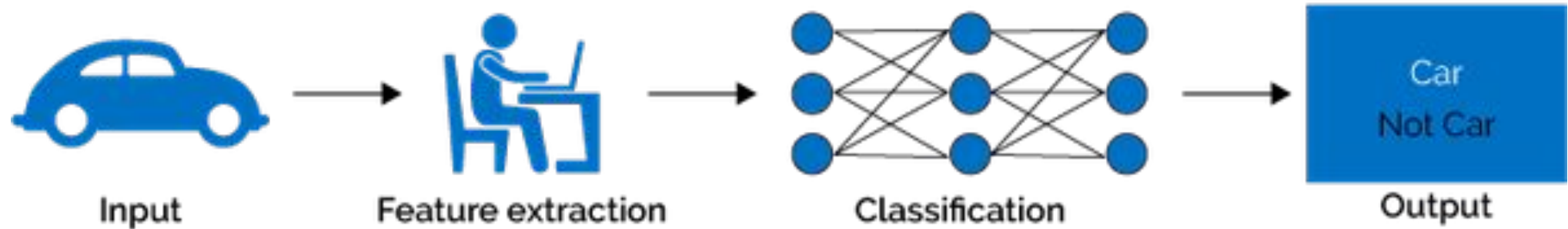
Applications



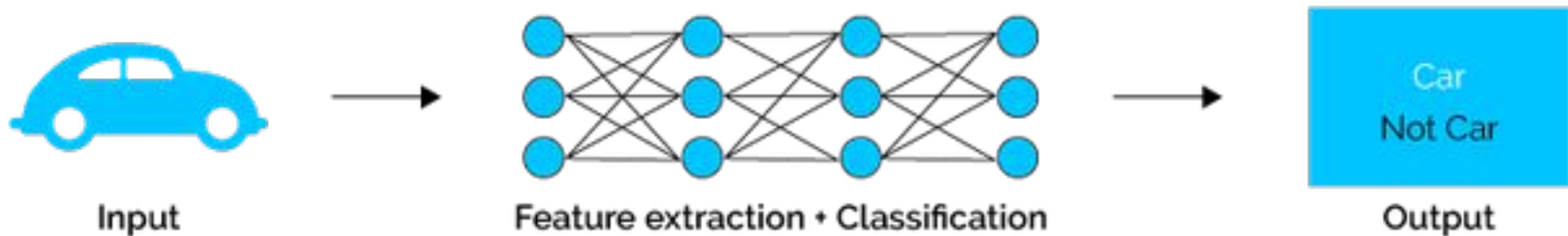
Neural Network



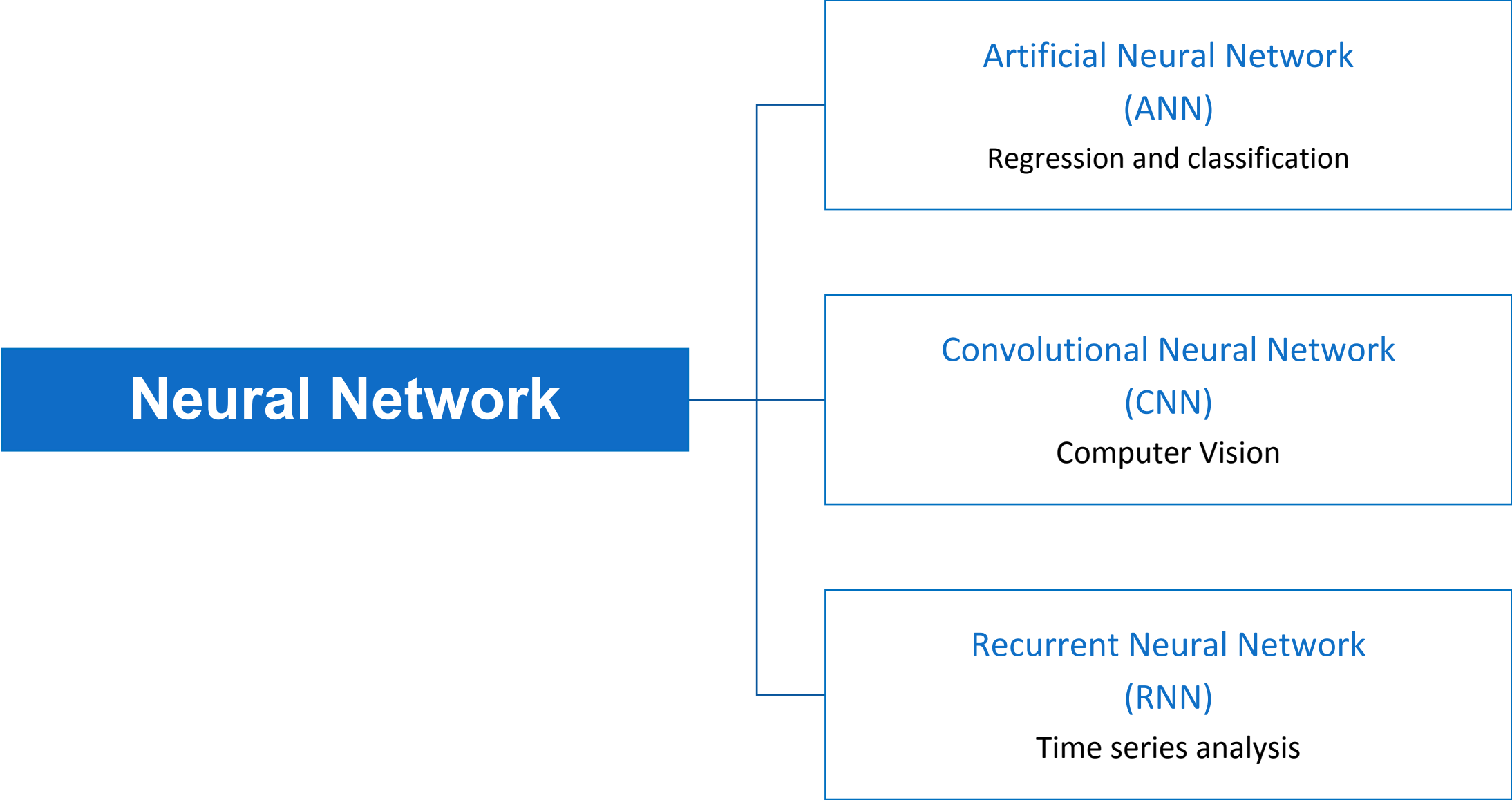
Shallow learning



Deep learning



Neural Network



```
graph LR; NN[Neural Network] --- ANN[Artificial Neural Network (ANN)]; NN --- CNN[Convolutional Neural Network (CNN)]; NN --- RNN[Recurrent Neural Network (RNN)];
```

Artificial Neural Network
(ANN)

Regression and classification

Convolutional Neural Network
(CNN)

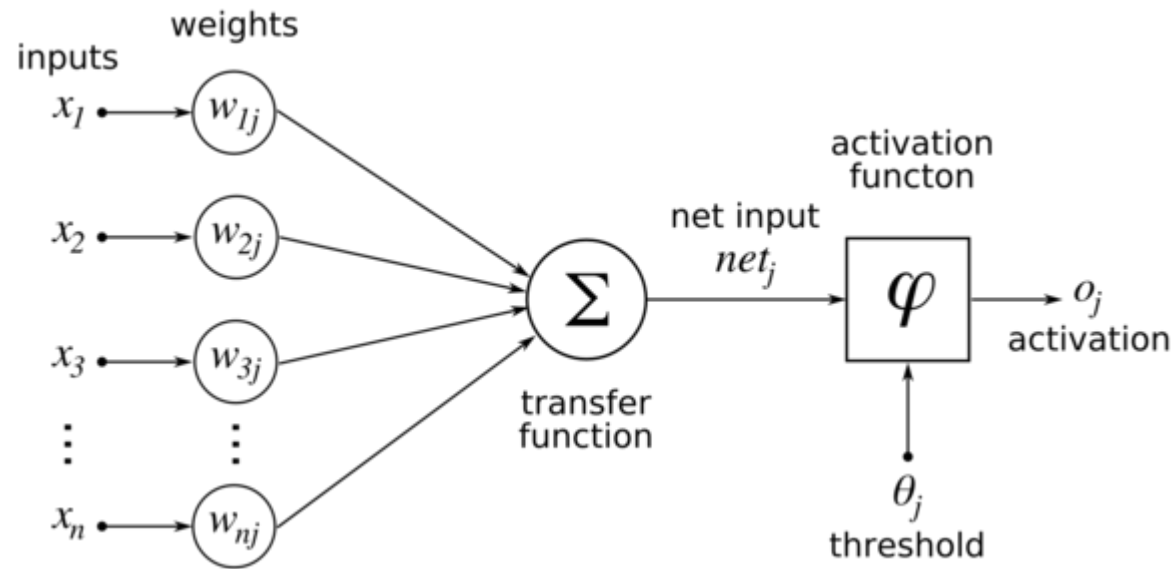
Computer Vision

Recurrent Neural Network
(RNN)

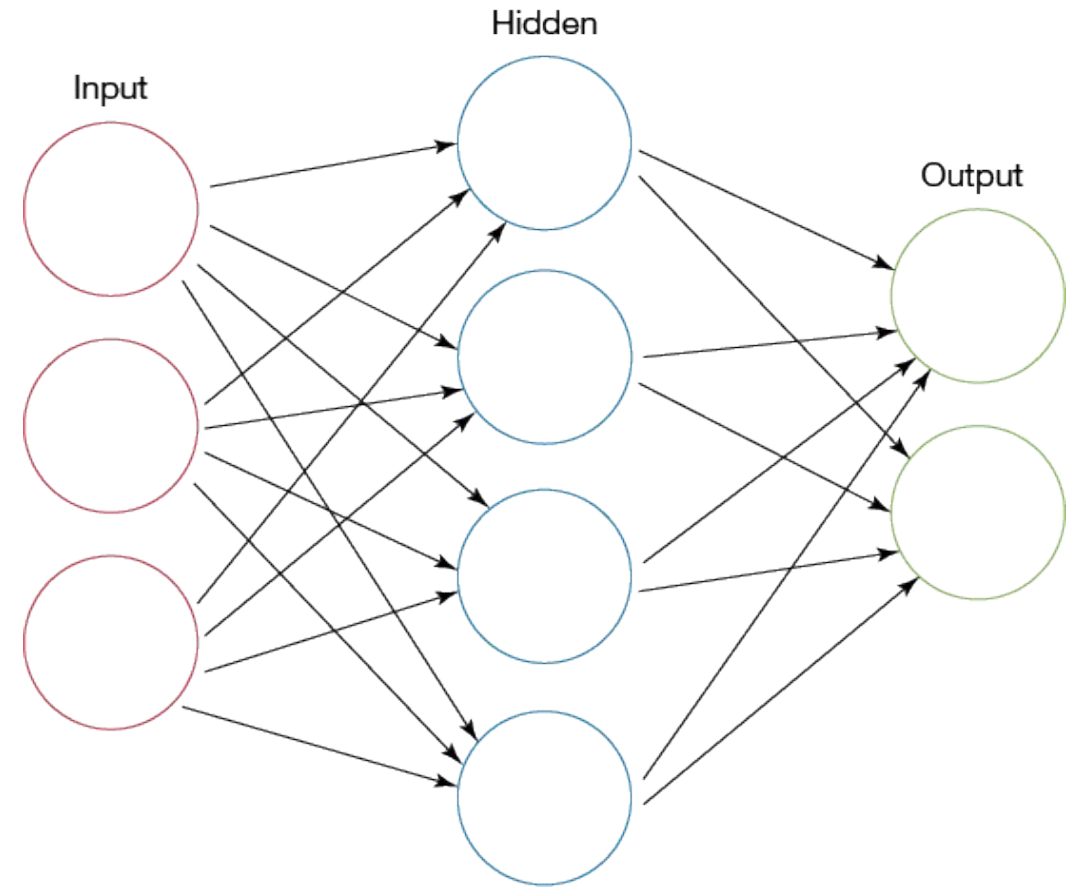
Time series analysis

Artificial Neural Network

Single layer perceptron



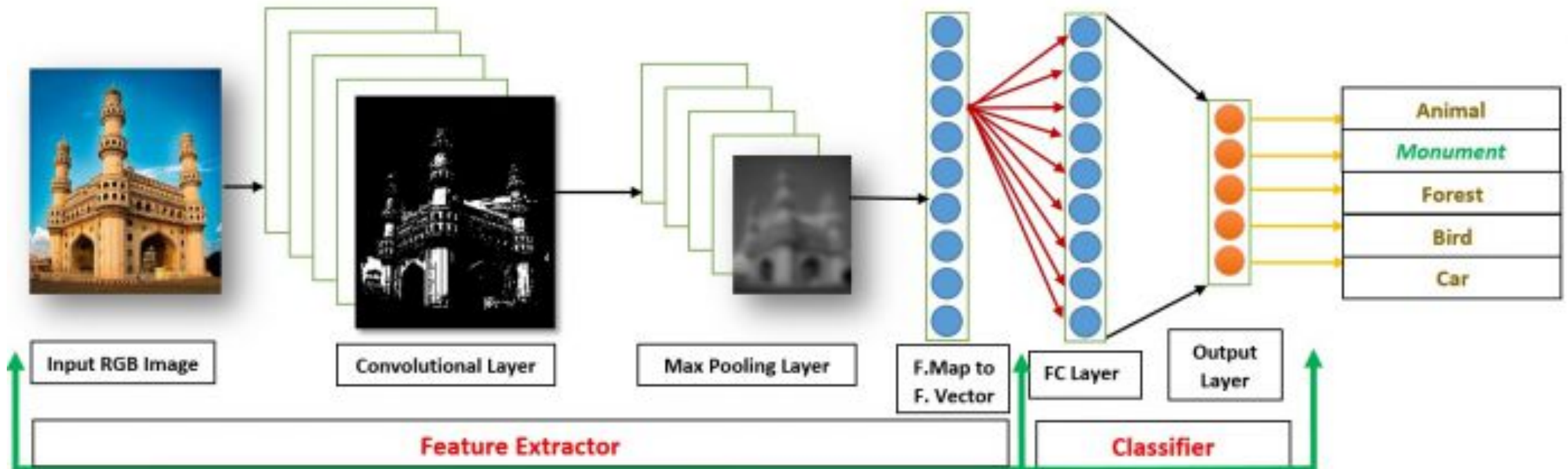
Multi-layer perceptron



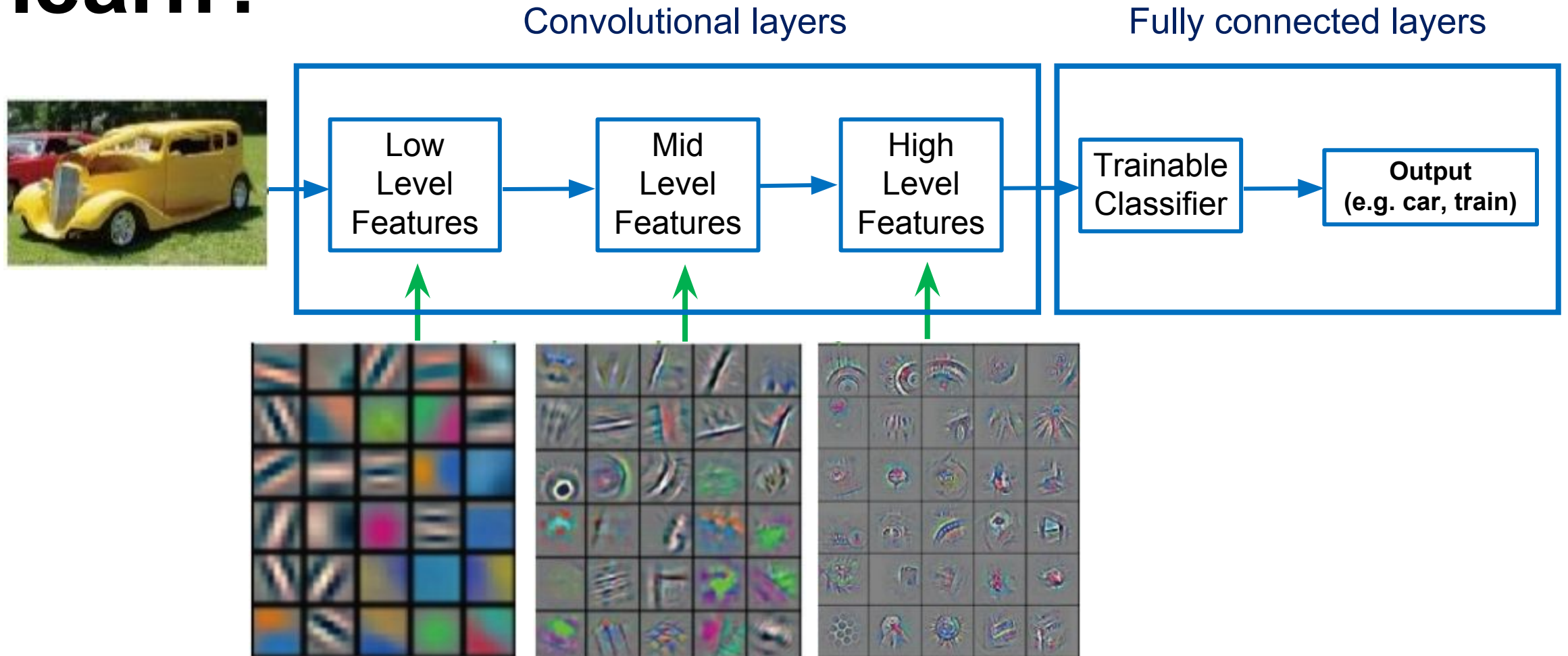
Convolutional Neural Network

Introduction

Convolutional Neural Networks (**CNNs**) learns **multi-level features** and **classifier** in a **joint fashion** and performs much **better than traditional approaches** for various image classification and segmentation problems.



CNN – What do they learn?

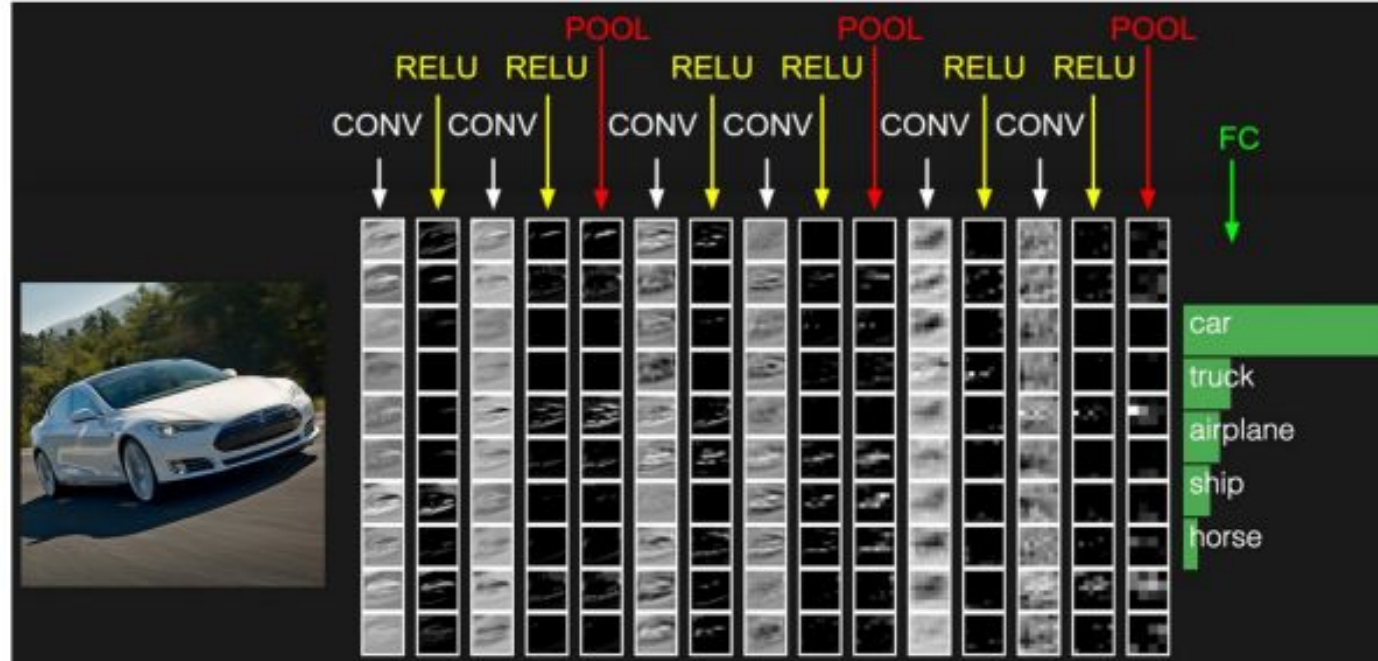


CNN -

Components

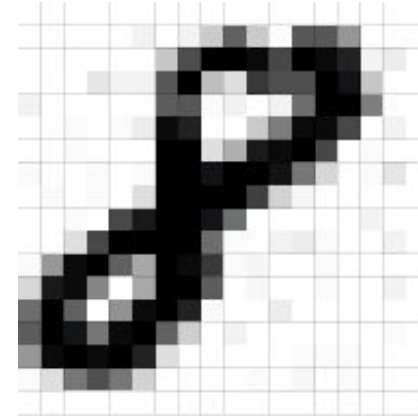
There are four main components in the CNN:

1. Convolution
2. Non-Linearity
3. Pooling or Sub Sampling
4. Classification (Fully Connected Layer)



Input

- An Image is a matrix of pixel values.
- If we consider a gray scale image, the value of each pixel in the matrix will range from 0 to 255.
- If we consider an RGB image, each pixel will have the combined values of R, G and B.



What We See

```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 24 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 24 97 17 78 78 94 83 14 88 34 89 63 72
21 36 23 09 75 00 74 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 47 15 94 03 80 04 62 16 14 09 53 54 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 24 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 35 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 47 48
```

What Computers See

Convolution

The primary purpose of Convolution in case of a CNN is to extract features from the input image.

1	0	1
0	1	0
1	0	1

Filter / Kernel / Feature detector

Image

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

Convolved Feature /
Activation Map /
Feature Map

Convolution...



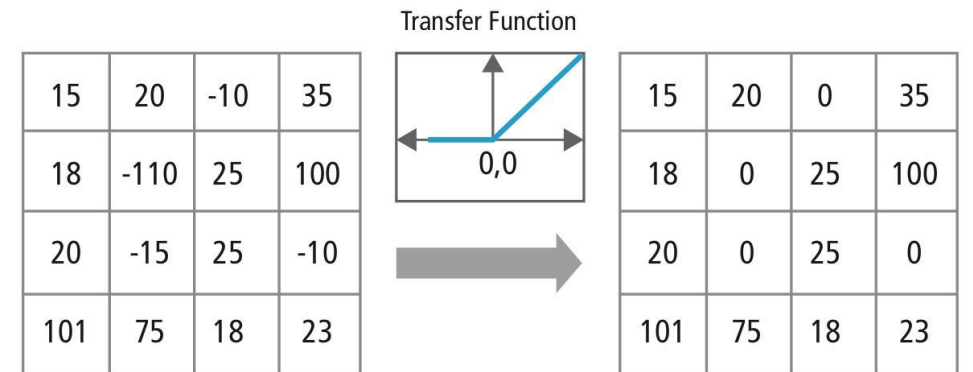
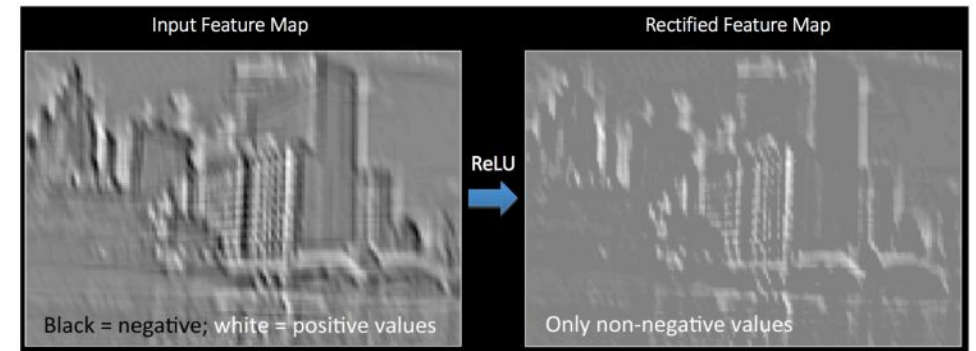
Input

Convolution...

- The size of the output volume is controlled by three parameters that we need to decide before the convolution step is performed:
 - ✓ **Depth:** Depth corresponds to the number of filters we use for the convolution operation.
 - ✓ **Stride:** Stride is the number of pixels by which we slide our filter matrix over the input matrix.
 - ✓ **Zero-padding:** Sometimes, it is convenient to pad the input matrix with zeros around the border, so that we can apply the filter to bordering elements of our input image matrix.
 - With zero-padding *wide convolution*
 - Without zero-padding *narrow convolution*

Non-Linearity (ReLU)

- Replaces all negative pixel values in the feature map by zero.
- The purpose of ReLU is to introduce non-linearity in CNN, since most of the real-world data would be non-linear.
- Other non-linear functions such as tanh $(-1,1)$ or sigmoid $(0,1)$ can also be used instead of ReLU $(0,\text{input})$.



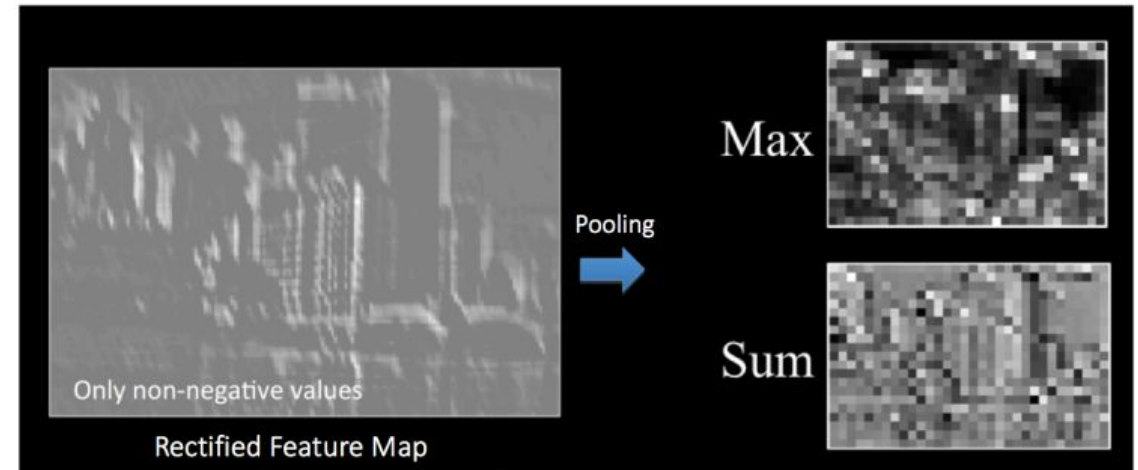
Pooling

Reduces the dimensionality of each feature map but retains the most important information. Pooling can be of different types: Max, Average, Sum etc.

1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

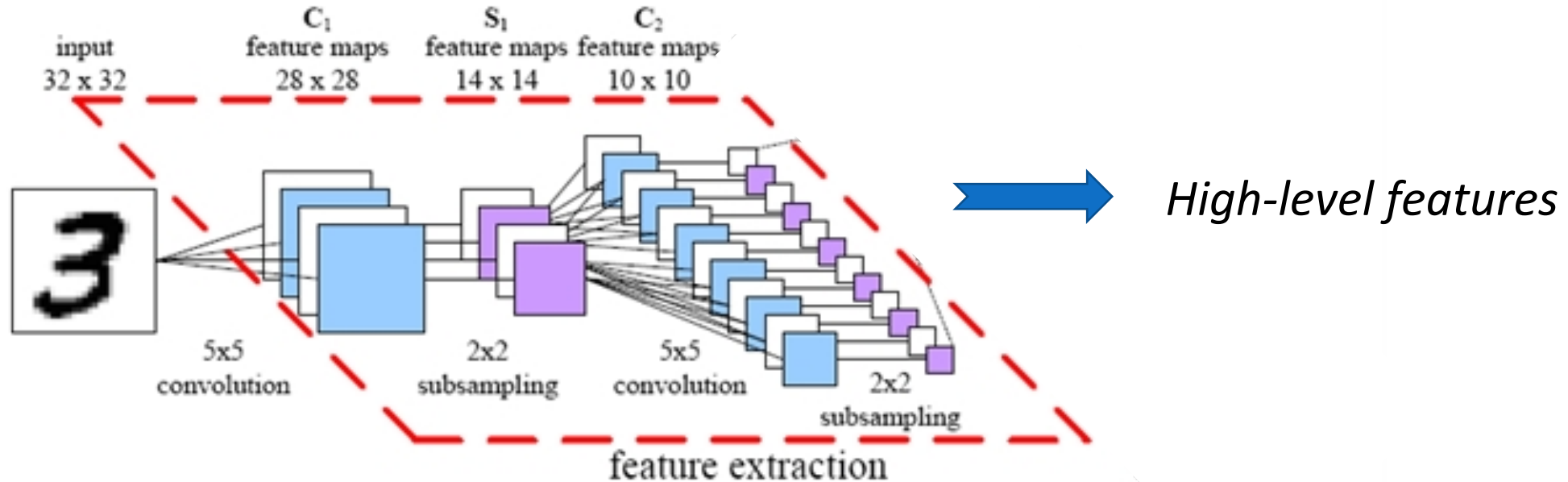
2x2 region

7	9
8	



Story so far

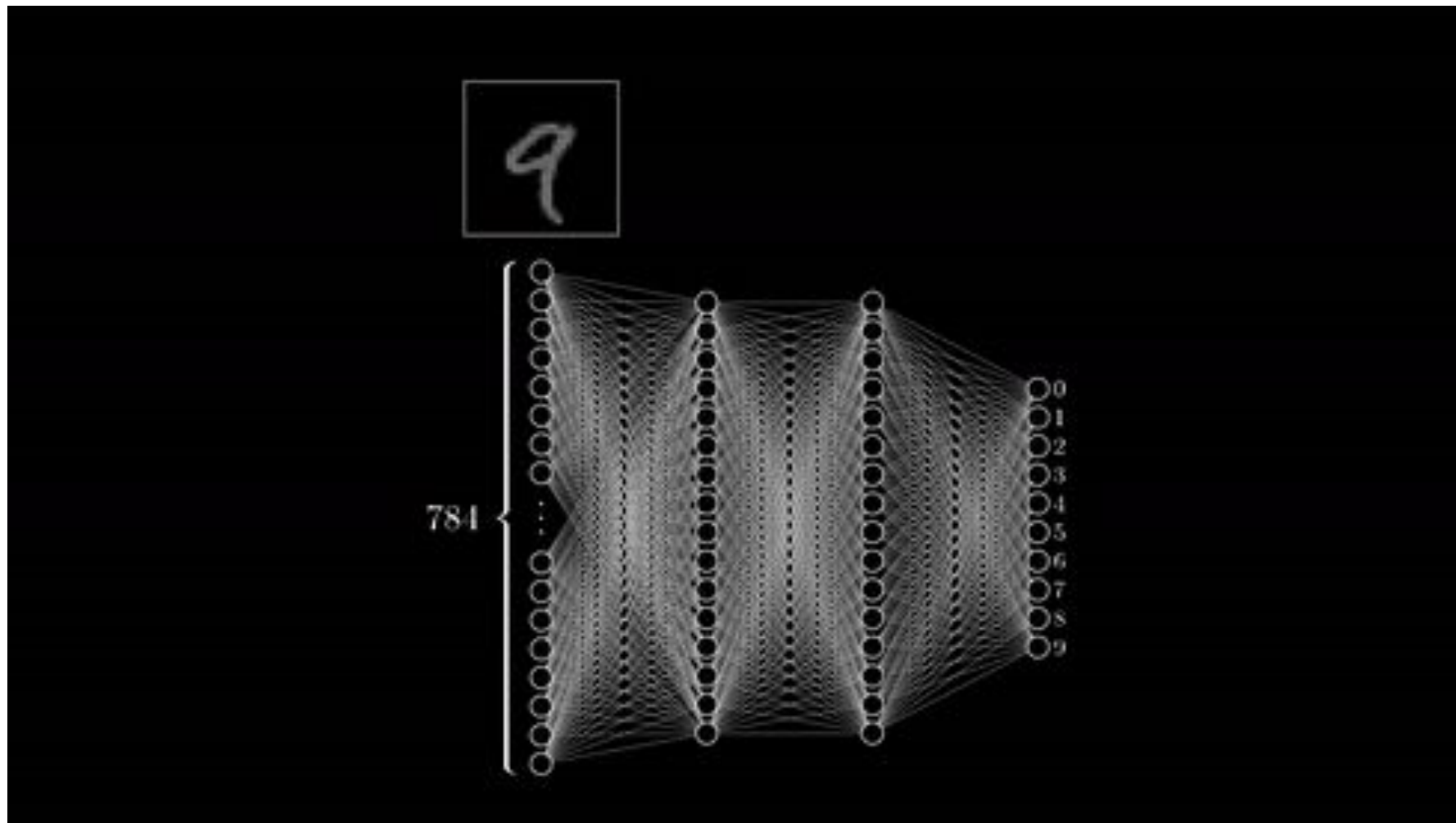
- Together these layers extract the useful features from the images.
- The output from the convolutional and pooling layers represent high-level features of the input image.



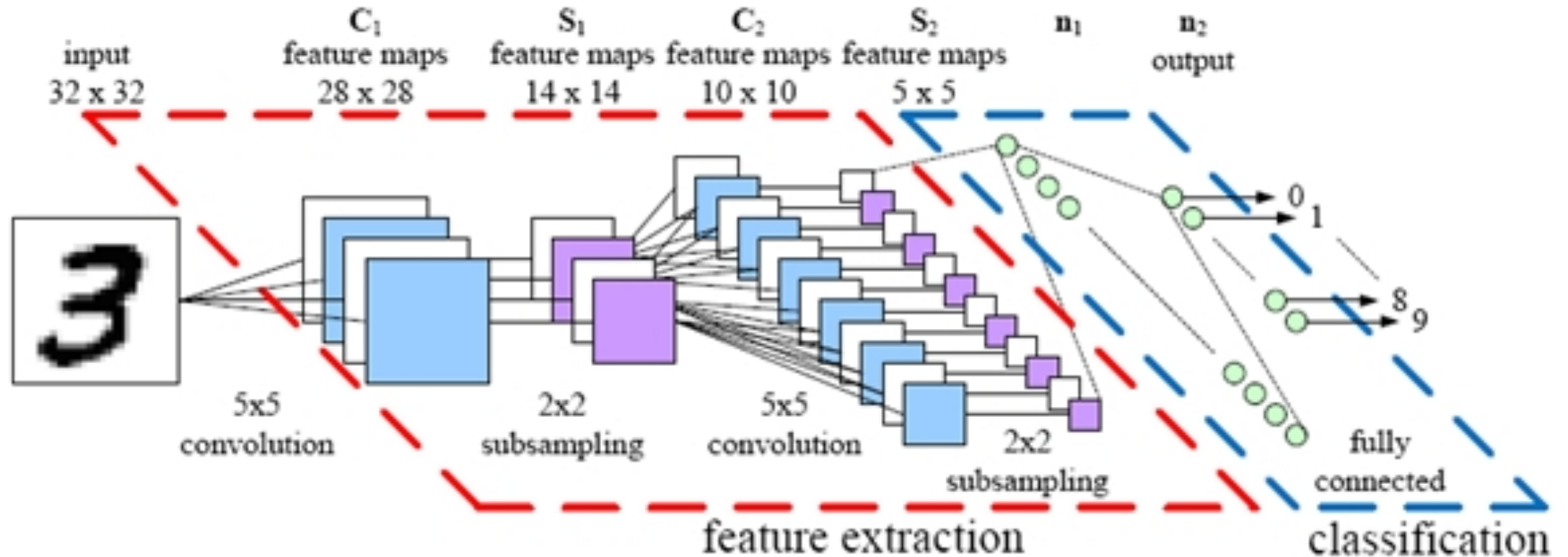
Fully Connected Layer

- A traditional Multi-Layer Perceptron.
- The term “Fully Connected” implies that every neuron in the previous layer is connected to every neuron on the next layer.
- Their activations can hence be computed with a matrix multiplication followed by a bias offset.
- The purpose of the Fully Connected layer is to use the high-level features for classifying the input image into various classes based on the training dataset.

Fully Connected Layer...



Overall CNN Architecture



Putting it all together – Training using Backpropagation

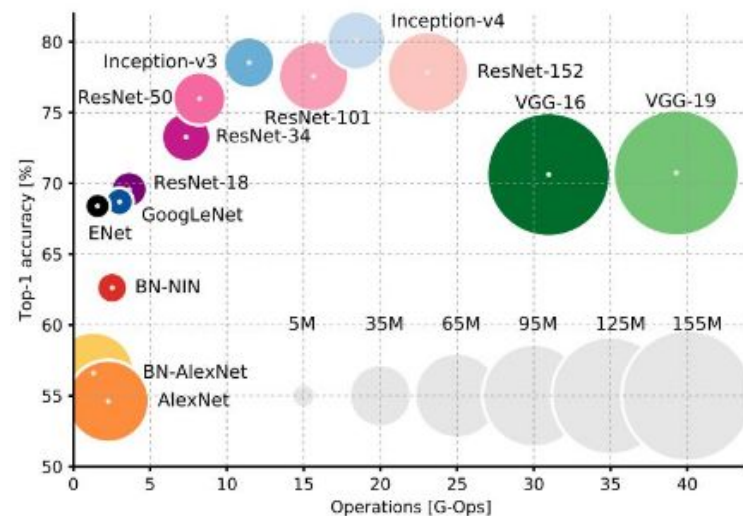
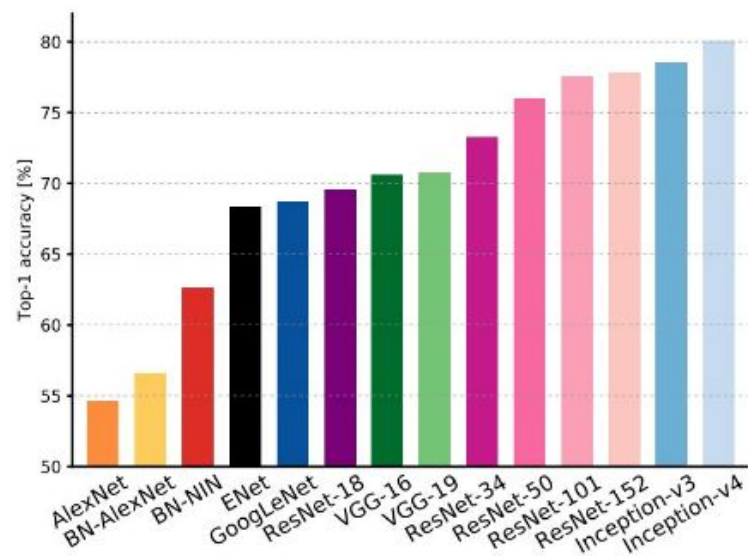
- **Step 1:** We initialize all filters and parameters / weights with random values.
- **Step 2:** The network takes a training image as input, goes through the forward propagation step (convolution, ReLU and pooling operations along with forward propagation in the Fully Connected layer) and finds the output probabilities for each class.
 - Let's say the output probabilities for the boat image above are [0.2, 0.4, 0.1, 0.3].
 - Since weights are randomly assigned for the first training example, output probabilities are also random.
- **Step 3:** Calculate the total error at the output layer (summation over all 4 classes).

- **Step 4:** Use Backpropagation to calculate the gradients of the error with respect to all weights in the network and use gradient descent to update all filter values/ weights and parameter values to minimize the output error.
 - The weights are adjusted in proportion to their contribution to the total error.
 - When the same image is input again, output probabilities might now be $[0.1, 0.1, 0.7, 0.1]$, which is closer to the target vector $[0, 0, 1, 0]$.
 - This means that the network has learnt to classify this particular image correctly by adjusting its weights / filters such that the output error is reduced.
 - Parameters like number of filters, filter sizes, architecture of the network etc. have all been fixed before Step 1 and do not change during training process – only the values of the filter matrix and connection weights get updated.
- **Step 5:** Repeat steps 2-4 with all images in the training set.

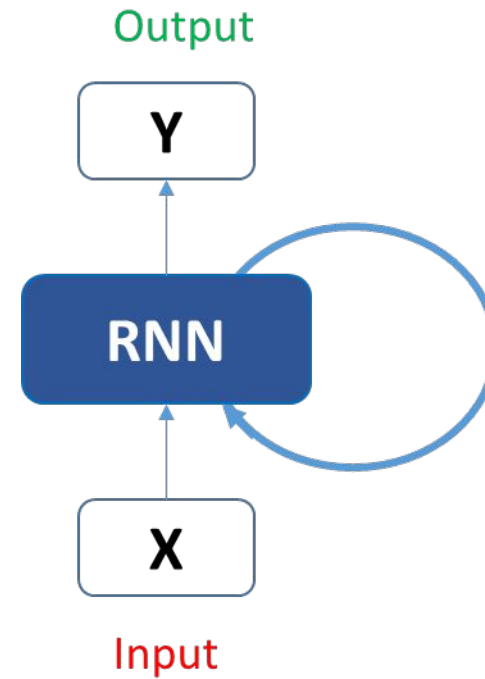
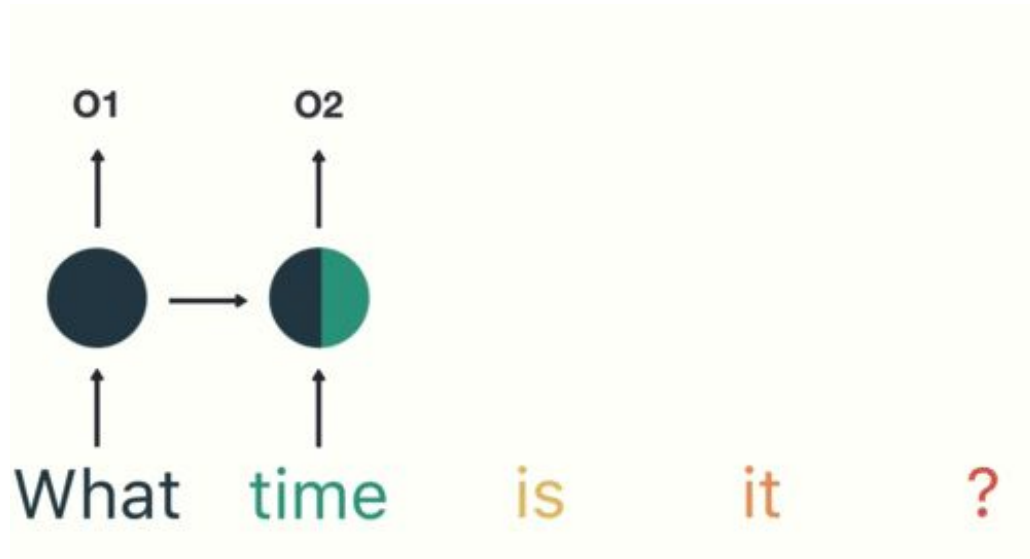
A thin vertical black line is positioned to the left of the title text.

CNN Architectures

Year	CNN Architecture	Developed By
1998	LeNet	Yann LeCun <i>et al.</i>
2012	AlexNet	Alex Krizhevsky, Geoffrey Hinton, and Ilya Sutskever
2013	ZFNet	Matthew Zeiler and Rob Fergus
2014	GoogleNet	Google
2014	VGGNet	Simonyan and Zisserman
2015	ResNet	Kaiming He
2017	DenseNet	Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger



Recurrent Neural Network - RNN



Visualizing a CNN

- **Adam Harley** created amazing visualizations of a Convolutional Neural Network trained on the MNIST Database of handwritten digits
 - 2D Visualisation of a CNN - <http://scs.ryerson.ca/~aharley/vis/conv/flat.html>
 - 3D Visualisation of a CNN - <http://scs.ryerson.ca/~aharley/vis/conv/>

Thank You!



